Plectis: Verifying Code and Recording Its Execution

Selected Dafny computations and their Python callers

Will Cook

October 2026

Abstract

Plectis verifies selected executable computations against explicit specifications and links them to their runtime callers and recorded executions. Its aim is to make a code guarantee understandable in the application that uses it. We describe the architecture through an ownership classifier that formerly gave unsafe advice when a stale claim preceded a live claim for the same file. The repaired computation gives live observations priority regardless of claim order, and the application calls the generated implementation. A separate replay-admission computation checks observations of component runs and their public records. The prototype contains 12 verified component computations and 4 further workflow and replay cores. We explain how specifications, generated code, application behavior and run evidence fit together, using the repair to show why a proof must be paired with review of the requirement it establishes. A snapshot-backed inspection API preserves these distinctions for a future frontend.

1 Aim and contribution

Formal verification is most useful when the proved computation is the one the application actually calls. Publishing a theorem beside a program does not by itself establish that connection. A developer must still identify the input model, follow the computation into its caller and understand which parts of the application remain outside the proof.

Plectis is a software prototype built around that problem. For selected local operations, it connects an explicit requirement to a verified computation, generated executable code, an application caller and a record of execution. The contribution is this connected implementation and an account of its boundaries. The ownership example below makes the connection concrete: a requirements review exposed an unsafe first-match rule, the specification and computation were repaired together, and the changed behavior was checked through the actual application path.

The wider catalogue contains 88 reference components. Formal coverage concerns 12 selected computations and 4 further workflow and replay cores. Each has a stated

AI assistance. Will Cook set the objectives and directed the work. Large-language-model agents produced and revised the software and manuscript.

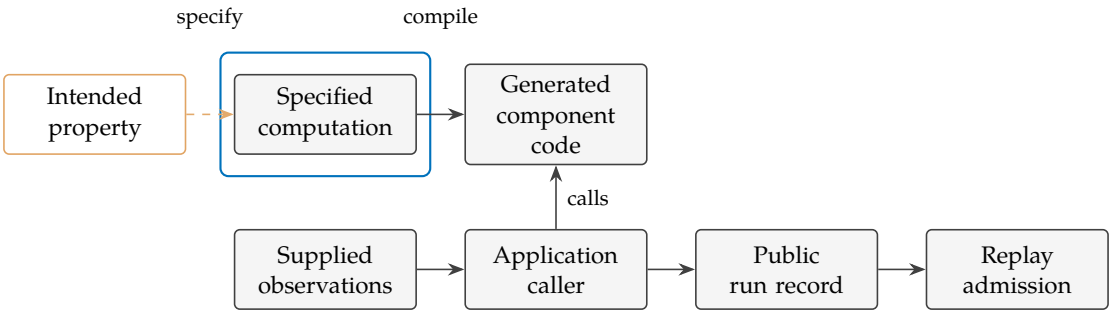

Figure 1: Upper row: state the intended property, verify the specified computation and compile its implementation. Lower row: the application calls that code on supplied observations and records its result; replay admission checks the recorded observations. The outline marks the computation's functional proof. Compilation, the caller and the observations have separate trust and test obligations.

property and input model; their presence does not make every operation in the catalogue verified. The public Plectis artifact and reproduction guide provides the implementation map, formal sources and reproduction instructions separately from this explanation.

2 Architecture: from a property to an observed run

Figure 1 separates two related activities. In the upper row, a requirement is expressed as a specification and an executable computation in Dafny. The verifier checks that the computation satisfies its contract. The compiler then produces Python for use by the application.

In the lower row, the application receives observations, normalizes them into the computation's input model and calls the generated code. It interprets the returned decision and records the public result. This caller matters: a correct finite computation would be irrelevant if the application supplied the wrong inputs, ignored the result or continued to call its previous implementation.

A separate admission computation checks observations of a selected set of runs. The host captures the declared routes, component identities, source and input identities, and the public payload intended for each record. It compares that payload with a separate read of what was saved. The raw returned result is kept separate because an application may publish a smaller public record than its full internal result. Admission therefore examines the relationship between a run's declared scope and its observations; it does not replace the component's own specification.

3 Worked behavior: protecting a live owner

The concurrency component advises what to do when generated files have drifted and supplied records identify their owners. Consider two claims for one file: an old session is observed stale, while an active session is observed live.

The previous classifier selected the first matching claim. When the stale row came first, it selected the old session and advised releasing or superseding that claim before regeneration. Reversing the rows selected the active session and advised waiting. Neither the input contract nor the validator required unique claims or newest-first ordering. An earlier stale observation could therefore hide a later live owner.

Order of the same observations	Previous decision	Repaired decision
Old, stale; then active, live	Release stale claim	Wait for active owner
Active, live; then old, stale	Wait for active owner	Wait for active owner

Table 1: Reversing the observations changes the first-match result but leaves the repaired decision unchanged. Both rows concern the same generated file.

The repaired requirement gives any matching live observation priority over stale or unresolved observations. If a generated file has a live claim, the aggregate decision must protect a live owner and advise against a sibling patch, regardless of claim order. Unresolved ownership has priority over stale ownership. Stale release is selected when at least one generated file has a stale owner and none has a live or unresolved owner; other files may be unowned.

The computation models paths and claims as finite sequences. For each generated path, it selects the earliest matching claim of highest freshness priority. It then combines the per-path results using the same protective ordering. The specification relates that selection to the final decision and action, including the requirement that the presence of a matching live claim forces live-owner protection. This is a statement about every finite input satisfying the model, rather than just the two rows in the table.

The application translates path strings and supplied freshness observations into that model. It retains conflicting observations and treats a missing owner identity as unresolved. After calling the generated computation, it selects a representative owner for the handoff. This last step must also select a live representative when the aggregate decision is live, even if an earlier stale owner belongs to another file.

Tests through the caller exercise reversed claim orders, claims on different files, unknown freshness, duplicate session observations and competing live sessions. The same-file cases retain live-owner protection in both orders. These checks address the application connection; the general finite-sequence property is established for the Dafny computation.

4 Verification method and evidence

The ownership repair illustrates two different questions. Verification asks whether the computation satisfies the specification. Requirements review asks whether that specification describes the intended behavior. A proof of the old first-match rule could have faithfully established an inadequate requirement. The counterexample made that inadequacy visible before the requirement and implementation were changed together.

The retained verification records report 236 verified tasks and rejection of 28 designated source mutants across the 12 component computations and 4 workflow and replay cores. Ownership classification records 9 tasks; navigation metrics records 31. These are verification-task counts, not a benchmark or a count of independent requirements.

The negative variants retain the specification while deliberately changing the implementation. For ownership, one restores first-match selection and another gives stale aggregate state priority over live. The verifier rejects both. Such challenges show that the selected mistakes are excluded by the current contract; they cannot establish that the contract captures every intended requirement.

Other components illustrate different local guarantees. An evidence-comparison computation partitions normalized identifiers into shared, missing and extra classes while retaining the final occurrence of a duplicate. Workflow cores address repair propagation, behavior-preserving replacement and public-only report transformation. Their guides state the particular model and property for each computation.

The replay-admission core has its own contract. It requires exactly one observation for each component in a nonempty declared selection, rejects unknown observations, and checks identities, status, required witness information and agreement of public payloads. Its proved decision is equivalent to that stated admission relation. It establishes a property of the supplied observations, not the truth of every domain claim made by a component.

The published records use Dafny 4.11.0. Native verification, reproduction of generated code, application tests and fresh replay are separate checks. Ordinary source and generated-byte comparisons detect drift without rerunning the prover. The Plectis artifact and reproduction guide names the toolchain, source objects and checks needed to repeat each part.

5 Reading the evidence through the backend

A reader needs to follow the connection from a component's stated purpose to its modeled contract, generated implementation and recorded runs. The backend provides that inspection path through a typed index. Component descriptions come from their maintained source, while the registry supplies identities and claim limits. The index joins these records to guides, replay configurations, formal subclaims, proof metadata and artifact references. It records absent artifacts explicitly instead of constructing links to material it does not have.

The local HTTP service validates this index once and creates an immutable snapshot. It then answers component searches, detail and coverage queries, and bounded graph-neighborhood requests from that snapshot. It does not read arbitrary files or invoke a component when a reader opens a page. Explicit replacement validates a new index before swapping it into service. Requests already in progress finish on their captured snapshot.

Each data response identifies its snapshot by a digest. Pagination binds its cursor to that revision, query and page size; a client must also compare revisions when combining a component with separately fetched proofs or records. This prevents a display from silently treating two different snapshots as one. The digest identifies the index, not

a Git revision or a new verification run. A machine-readable API contract and client implementation guide specify these loading rules and the fields a frontend can consume. The interface itself is future client work; it is not part of the proved computation.

The distinction between evidence classes must survive presentation. A retained proof result, a source-byte comparison, a host test and a fresh component run answer different questions. The backend preserves those fields separately and leaves whole-claim coverage unknown when it has not been mapped. For example, an ownership component may expose a proved priority rule and its caller links without establishing that today's observed owner is authentic or still live. A frontend should make that combination visible, rather than reducing it to a single verified-component label. Transport and contract tests check the reading interface; they do not extend the formal guarantees described above.

6 Context and related work

Dafny provides specifications and automated verification for executable computations [1]. IronFleet combines state-machine refinement and implementation verification for distributed systems [2]. Plectis uses a smaller unit: selected local computations and their connections to Python callers, rather than a verified distributed runtime.

IronSpec studies specifications that verify but miss developer intent [3]; MutDafny uses implementation mutations to expose weaknesses in Dafny specifications [4]. The ownership counterexample and selected negative variants address this same distinction between satisfying a contract and choosing an adequate contract. Plectis has not adopted either framework, and rejection of its selected mutants is not a completeness result.

CompCert establishes a semantic-preservation property for compilation to machine code [5]. Plectis trusts its Dafny-to-Python compiler and checks the identity of generated artifacts. Anthropic and Theorem report Rocq proofs for selected compiled RISC-V fuzz targets under explicit Linux-process models and bounds [6, 7]. Those proofs have a different target and trusted base from Plectis's source computations and Python hosts. We make no numerical coverage comparison or end-to-end equivalence claim.

7 Boundaries

The formal guarantees concern the stated finite models. Their adequacy remains a requirements question. Python normalization, authenticity of freshness, path-scope relationships, filesystem observations, public-record projection, compilation and the execution platform remain trusted or separately tested. An observed owner label does not prove a real lock or a session's liveness.

Dafny, Boogie and Z3 are part of the verification chain; these SMT-backed results are not independently kernel-checked Lean or Rocq proof objects. Application tests and run records supplement the component proofs without extending their scope to the whole catalogue. The prototype does not establish production readiness or an independent evaluation.

References

[1] K. R. M. Leino. *Dafny: An Automatic Program Verifier for Functional Correctness*. LPAR-16, LNCS 6355, 2010, pp. 348–370. Project and documentation.

[2] C. Hawblitzel et al. *IronFleet: Proving Practical Distributed Systems Correct*. SOSP, 2015. Publication.

[3] E. Goldweber et al. *IronSpec: Increasing the Reliability of Formal Specifications*. OSDI, 2024, pp. 875–891. Paper and talk.

[4] I. Amaral, A. Mendes and J. Campos. *MutDafny: A Mutation-Based Approach to Assess Dafny Specifications*. ICSE, 2026. arXiv:2511.15403.

[5] X. Leroy. *Formal verification of a realistic compiler*. Communications of the ACM 52(7), 2009, pp. 107–115. Paper.

[6] K. Lucas et al. *LLMs are making formal software verification feasible at a much larger scale*. Anthropic Frontier Red Team, 22 October 2026. Research report.

[7] Theorem. *OSS-Verify proofs*. Public statements, provenance and Rocq proofs for compiled fuzz targets, 2026. Repository.